

Lab 05 — Praktijklab: van JDK naar JRE, van Node naar nginx

Doel: dezelfde API twee keer bouwen — in één stap en dan multi-stage — het verschil meten, hetzelfde doen voor een "Angular"-frontend, en dan buildcaches en een image zonder shell aanraken.

Vereisten — Lab 04 afgewerkt: `~/labo-docker/04/Api.java` bestaat, de images `eclipse-temurin:21-jdk` en `eclipse-temurin:21-jre-alpine` zijn aanwezig.

Geleverde bestanden (`files/`) - `web/package.json` en `web/src/index.html` — een nep-Angular-project. De "build" wordt gesimuleerd door een `cp`: we bootsen de **vorm** van een frontend-project na, niet de inhoud.

Je schrijft elke Dockerfile zelf: dat is de oefening.

Stap 1 — De "alles-in-één"-image

```
mkdir -p ~/labo-docker/05 && cd ~/labo-docker/05
cp ~/labo-docker/04/Api.java .
```

Maak `Dockerfile.mono` aan — compilatie **en** uitvoering in dezelfde image:

```
FROM docker.io/library/eclipse-temurin:21-jdk
WORKDIR /app
COPY Api.java .
RUN mkdir -p build && javac -d build Api.java && jar --create --file api.jar --main-class Api -C build .
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -f Dockerfile.mono -t api-mono:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-mono|temurin'
```

Observeer `localhost/api-mono 1.0 488 MB` — precies de grootte van `eclipse-temurin:21-jdk`. De JAR van 2 KB heeft niets toegevoegd; de JDK heeft alles meegebracht.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version'
```

Observeer `Api.java`, `api.jar`, `build`, en `javac 21.0.x`: de broncode **en** de compiler zitten in de productie-image.

Uitleg. Deze image werkt perfect. Dat is wat ze gevaarlijk maakt: niets wijst erop dat 480 MB tooling en je bronnen met elke uitrol meegaan.

Stap 2 — De multi-stage

Maak `Dockerfile` aan:

```
# ----- stage 1: build -----
FROM docker.io/library/eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Api.java .
RUN mkdir -p build && javac -d build Api.java && jar --create --file api.jar --main-class Api -C build .

# ----- stage 2: runtime -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /src/api.jar /app/api.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

```
podman build -t api-multi:1.0 .
```

Observeer de voorvoegsels `[1/2] STEP 1/4 ...` en dan `[2/2] STEP 1/6 ...`: Buildah nummert de stages, en alleen de laatste eindigt met `COMMIT api-multi:1.0`.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-mono|api-multi'
podman run --rm --entrypoint sh api-multi:1.0 -c 'ls /app; javac -version'
podman run --rm --entrypoint ls api-multi:1.0 /src
```

Observeer `209 MB` tegenover `488 MB`, dan `api.jar` alleen, `sh: javac: not found`, en `ls: cannot access '/src': No such file or directory`.

Uitleg. De stage `build` bestond zolang de compilatie duurde, en werd dan weggegooid. De uiteindelijke image kent er alleen het bestand van dat door `COPY --from` gekopieerd is. Geen bronnen, geen JDK, geen map `/src`: ze hebben nooit deel uitgemaakt van haar lagen.

```
podman history api-multi:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6
```

Observeer een laag van `4.61kB` voor de `COPY` van de JAR: al de rest komt van de basisimage.

VALKUIL

`COPY --from=build /src /app` zou de hele map gekopieerd hebben, `Api.java` en `build/` inbegrepen. Je kopieert **het artefact**, niet de werkmap. Dat is vraag 3.

Stap 3 — Binnenkijken in een stage

Een weggegooid stage is niet inspecteerbaar... tenzij je vraagt daar te stoppen:

```
podman build --target build -t api-build-stage .
podman run --rm api-build-stage ls -la /src
```

Observeer `Api.java`, `api.jar`, `build/`: dat is de exacte inhoud van de stage op het moment dat stage 2 er `api.jar` uit kopieerde.

Uitleg. `--target` is het diagnosegereedschap van multi-stage: wanneer een `COPY --from` faalt met `no such file or directory`, bouw je de stage alleen en kijk je, in plaats van paden te raden (vraag 12).

```
podman rmi api-build-stage
```

Stap 4 — De frontend: Node bouwt, nginx serveert

```
mkdir -p web && cp -r <pad-van-het-lab>/files/web/* web/  
ls -R web
```

Maak `web/Dockerfile` aan:

```
FROM docker.io/library/node:22-alpine AS build  
WORKDIR /app  
COPY package.json .  
RUN echo "npm ci (gesimuleerd)"  
COPY src ./src  
RUN mkdir -p dist/web/browser && cp src/index.html dist/web/browser/ && echo "ng build  
(gesimuleerd)"  
  
FROM docker.io/library/nginx:alpine  
COPY --from=build /app/dist/web/browser /usr/share/nginx/html  
EXPOSE 80
```

```
podman build -t web-multi:1.0 web  
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'web-  
multi|node|nginx'
```

Observeer `web-multi 1.0 64.2 MB` — exact de grootte van `nginx:alpine` — terwijl `node:22-alpine` `167 MB` weegt.

```
podman run -d --name web -p 18081:80 web-multi:1.0  
curl -s localhost:18081/  
podman rm -f -t 0 web
```

Observeer je `index.html`, geserveerd door nginx. Open ook `http://localhost:18081/` in de Windows-browser.

Uitleg. Node diende om te "bouwen" (hier vervangt een `cp` de `ng build`), en verdween dan. De frontend in productie is een statische bestandsserver: daarom bevat de `web`-image van een Angular-stack nooit Node.

→ ANGULAR

Vervang op een echt project `RUN echo "npm ci (gesimuleerd)"` door `RUN npm ci` en de `cp` door `RUN npm run build`, en kopieer `dist/<projectnaam>/browser`. De scheiding `COPY package*.json → npm ci` → `COPY . .` is die van de cache van lab 04: de 900 MB van `node_modules` worden alleen opnieuw gedownload als `package-lock.json` verandert.

Stap 5 — Een cache die builds overleeft

Maak `Dockerfile.cache` aan:

```
FROM docker.io/library/eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Api.java .
RUN --mount=type=cache,target=/root/.m2 sh -c 'echo "dep-$(date +%s)" >> /root/.m2/marker; cat
/root/.m2/marker' \
&& mkdir -p build && javac -d build Api.java

FROM docker.io/library/alpine
COPY --from=build /src/build /app
```

De `marker` simuleert de Maven-repository `~/m2`: elke build voegt er een regel aan toe.

```
podman build --no-cache -f Dockerfile.cache -t cache-demo . 2>&1 | grep dep-
podman build --no-cache -f Dockerfile.cache -t cache-demo . 2>&1 | grep dep-
```

Observeer één regel `dep-...` bij de eerste build, **twee** bij de tweede — terwijl `--no-cache` alles herbouwd heeft. De map `/root/.m2` heeft het tussen de twee builds overleefd.

```
podman run --rm cache-demo ls /root/.m2
```

Observeer `ls: /root/.m2: No such file or directory`: de cache zit **niet** in de image.

Uitleg. De *cache mount* is een map die Buildah in je gebruikersopslag bijhoudt, gemount in de buildcontainer voor de duur van één instructie. Op een echt project vermijdt `RUN --mount=type=cache,target=/root/.m2 mvn package` dat bij elke build 300 MB afhankelijkheden opnieuw gedownload worden — zelfs als `pom.xml` verandert. Geen enkele regel `# syntax=` was nodig: Buildah begrijpt `--mount` van nature.

Stap 6 — musl of glibc?

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --
version 2>&1 | head -1; head -1 /etc/os-release'
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version 2>&1
| head -1; head -1 /etc/os-release'
```

Observeer `musl libc (x86_64)` / `Alpine Linux` aan de ene kant, `ldd (Ubuntu GLIBC 2.xx)` / `Ubuntu` aan de andere.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'apk info |
wc -l'
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'dpkg -l | grep -c
^ii'
```

Observeer ongeveer `73` pakketten tegenover `140`.

Uitleg. Dit is het commando dat je uitvoert **vóór** je een image naar Alpine migreert: als een native bibliotheek van je applicatie voor `glibc` gecompileerd is, laadt ze niet met `musl`. Het aantal pakketten is dan weer wat een kwetsbaarheidsscanner telt: half zoveel pakketten, half zoveel mogelijke CVE's.

Stap 7 — Distroless: helemaal geen shell

```
podman pull gcr.io/distroless/java21-debian12
podman images --format '{{.Repository}} {{.Size}}' | grep distroless
```

Observeer `194 MB` — minder dan de Alpine-JRE, hoewel het Debian is.

Maak `Dockerfile.distroless` aan:

```
FROM gcr.io/distroless/java21-debian12
COPY --from=localhost/api-multi:1.0 /app/api.jar /app/api.jar
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

```
podman build -f Dockerfile.distroless -t api-distroless:1.0 .
podman run -d --name d -p 18082:8080 api-distroless:1.0
sleep 2 ; curl -s localhost:18082/actuator/health ; echo
podman exec d sh -c 'ls'
podman exec d id
podman rm -f -t 0 d
```

Observeer `{"status":"UP"}` — de API draait — en dan `executable file sh not found in $PATH` en dezelfde fout voor `id`: er zit **niets** in deze image behalve Java en je JAR.

Uitleg. `COPY --from=` aanvaardt ook de naam van een bestaande **image**, niet alleen een stage. En een image zonder shell is een image waarin een aanvaller die code kan uitvoeren `sh`, `curl` noch `apt` vindt — maar waar jij ook niet in kunt. Je compenseert met rijke logs, `/actuator`, en `podman cp` om een bestand uit te halen.

Opruimen

```
podman rmi api-mono:1.0 api-multi:1.0 web-multi:1.0 cache-demo api-distroless:1.0 \
    gcr.io/distroless/java21-debian12 docker.io/library/node:22-alpine 2>/dev/null
podman rmi $(podman images --filter dangling=true -q) 2>/dev/null
podman images --format '{{.Repository}}:{{.Tag}}'
```

Observeer dat `alpine`, `nginx:alpine`, `eclipse-temurin:21-jdk`, `eclipse-temurin:21-jre` en `eclipse-temurin:21-jre-alpine` overblijven. Bewaar `~/labo-docker/05/Dockerfile`: de stack van de volgende labs zal hem gebruiken.

Wat je nu moet kunnen beweren

- Een mono-stage image weegt zoveel als haar tooling: 488 MB voor een JAR van 2 KB.

- Multi-stage behoudt alleen het gekopieerde artefact: 209 MB, zonder bronnen of compiler — en `--target` laat toe een stage te inspecteren.
- De Angular-frontend in productie is een nginx-image van 64 MB; Node zit er niet in.
- Een *cache mount* overleeft builds en komt niet in de image; Buildah beheert hem zonder `#syntax=`.
- Alpine = `musl`, Ubuntu = `glibc`: `ldd --version` zegt het, en een test valideert het.
- Distroless: `{"status":"UP"}` maar geen `sh` — beveiliging tegenover observeerbaarheid.